

Homework 3

1 Graph Isomorphism

- Graph neural networks update node embeddings through the computation graph defined by the neighborhood. **Question:** Draw the computation graph of *Node 1* in *Graph 1* in Figure 1 for 2-layer GNNs (i.e., aggregate neighbor nodes twice).

Note: you can draw in any way you like, take a picture or a screenshot to insert the figure into your submitted answer.

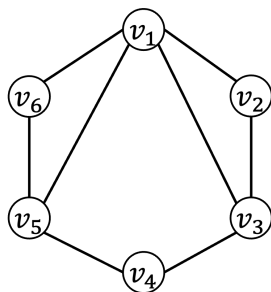


Figure 1: Graph 1

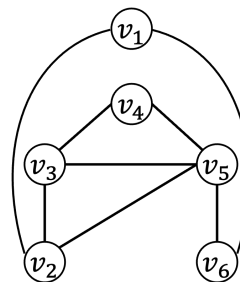


Figure 2: Graph 2

- Question:** If every node in *Graph 1* has the same initial features, and no IDs will be seen by GNNs, which pairs of nodes cannot be distinguished by GNNs? Explain why.
[Option] Is there any method to make the GNNs distinguish between these node pairs?
- In graph theory, graph G and graph H are **isomorphic** if there is a bijection between the vertex sets of G and H : $f : V(G) \rightarrow V(H)$ such that any two vertices u and v of G are adjacent in G if and only if $f(u)$ and $f(v)$ are adjacent in H .

Question: Are *Graph 1* in Figure 1 and *Graph 2* in Figure 2 isomorphic? If so, demonstrate the bijection function f between *Graph 1* and *Graph 2*. If not, prove it by finding a structure in one graph but is not present in the other. (**Hint:** use color refinement algorithm to extract the structure of K -hop neighborhood. Two isomorphic graphs should share the same K -hop neighboring information for any K .)

2 Contrastive Learning

Contrastive learning refers to a powerful class of embedding methods, and is widely used in self-supervised learning scenarios [1, 2]. The fundamental idea behind the contrastive learning paradigm is to encourage similar pairs of input samples to be represented close to each other, while

dissimilar ones are pushed far apart. Contrastive learning can be applied in settings where it is possible to construct similar and dissimilar pairs in a training dataset (a process known as data augmentation). For example, in computer vision, an image can be rotated to generate a 'similar' pair. For graphs, pairs can be existing nodes: often, graphs have a small proportion of positive pairs (ex. neighboring nodes) and a much larger proportion of negative pairs (ex. all pairs of nodes without links between them). Negative sampling is a common technique to improve training efficiency for large graphs, whereby a small subset of negative examples is randomly selected for each positive example.

1. Let's first apply the contrastive learning paradigm to a multi-way classification task: we have a list of input samples $\{x_i\}$, each with a corresponding label $\{y_i\}$ among k classes. We would like to learn a function f that encodes each input x_i into an embedding vector $f(x_i)$, such that data from the same class have similar embeddings and data from different classes have very different ones. This encourages higher quality embeddings that are more separable, thereby allowing for easier classification.

The specific function f could be of any form (e.g. linear function or MLP); for now, we will keep it unspecified. A standard contrastive loss function is prescribed as

$$\mathcal{L} = \sum_i \sum_j \mathbb{1}[y_i = y_j](1 - s(f(\mathbf{x}_i), f(\mathbf{x}_j))) + \mathbb{1}[y_i \neq y_j] \max(0, \epsilon - (1 - s(f(\mathbf{x}_i), f(\mathbf{x}_j)))) \quad (1)$$

where $\mathbb{1}[x] = 1$ if x evaluates to true and 0 otherwise, and $s(\mathbf{z}_i, \mathbf{z}_j) = \frac{\mathbf{z}_i \cdot \mathbf{z}_j}{\|\mathbf{z}_i\| \cdot \|\mathbf{z}_j\|}$ is the cosine similarity metric. Note that $\epsilon > 0$. **Question:** In a few sentences, explain how this loss function accomplishes the desired goal.

2. **Question:** In the equation above, what is the significance of the hyperparameter ϵ ?
3. To apply this paradigm to a graph setting, we need to first define what 'similar' means for graph learning.

For now, let us first consider node-prediction tasks. The similarity function $s_{\mathcal{G}}(v_i, v_j)$ should have a high value for similar nodes, and a low value for dissimilar nodes. Here v_i and v_j refer to the two nodes of interest. Some examples include:

- $s_{\mathcal{G}}(v_i, v_j) = \mathbf{A}_{ij}$, entries of the adjacency matrix
- $s_{\mathcal{G}}(v_i, v_j) = \mathbf{A}_{ij}^2 + \mathbf{A}_{ij}$, incorporating 2-hop connections.
- $s_{\mathcal{G}}(v_i, v_j) = \frac{|N(v_i) \cap N(v_j)|}{|N(v_i) \cup N(v_j)|}$, Jaccard's Coefficient, where $N(v_i)$ is the set of direct (1-hop) neighbors of vertex v_i .

Question: For the following graph (Figure 3), compute each of the above similarity measures for:

- (a) Vertex A and Vertex C
 - (b) Vertex B and Vertex E
 - (c) Vertex C and Vertex D
4. The contrastive learning loss function from Eq. 1 uses class labels (y_i) to define positive and negative pairs. However, in unsupervised graph learning settings, one must use "self-supervised" metrics of similarity, such as those in part 3 of this question. Then, one must

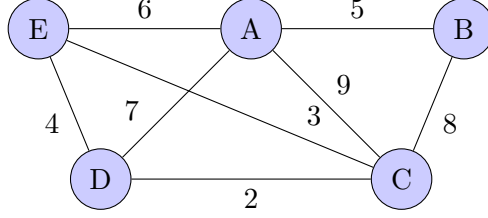


Figure 3: Example Graph for Similarity Metrics. Note: ignore the edge weights for the purposes of this exercise, although there are certainly similarity metrics that incorporate edge weight.

craft “similar” and “dissimilar” pairs of nodes. One possible method to do this is to first define a similarity threshold for s_G , denoted as γ . **Question:** Use γ to craft an unsupervised loss function for contrastive learning of node embeddings by modifying the loss function in Eq. 1.

5. Contrastive loss functions might help combat the phenomenon of oversmoothing in graph neural networks (ex. (Chen et al.)), wherein node embeddings become similar. Oversmoothing can result from excessively deep GNN architectures, or from inadequate loss functions. **Question:** In a sentence or two, explain how the contrastive loss function you devised in the last part might mitigate oversmoothing of the second kind.

3 Graph Transformer Positional Encoding

Recent studies on graph transformer positional/structural encoding mainly focus on (1) how to utilize local/global topological information to construct positional/structural encoding for GTs and (2) how to design effective and efficient methods to incorporate PE/SE into GTs, either in combination with a GNN or inserting PE/SE into attention scores.

1. **Laplacian Positional Encoding.** Given a simple undirected graph with adjacency matrix A as:

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix} \quad (2)$$

The normalized graph Laplacian L is a matrix representation of a graph that captures its structure. It is derived from the degree matrix D and the adjacency matrix A of the graph.

The normalized graph Laplacian L is defined as:

$$L = I - D^{-\frac{1}{2}} A D^{-\frac{1}{2}}$$

Question: Compute the degree matrix D and the normalized graph Laplacian L .

2. Given the normalized Laplacian matrix L of a graph, if λ is an eigenvalue of L and x is its corresponding eigenvector, then:

$$Lx = \lambda x \quad (3)$$

To find the eigenvalues, we set up and solve the characteristic polynomial:

$$\det(L - \lambda I) = 0 \quad (4)$$

where $\det$ denotes the determinant, I is the identity matrix, and λ represents potential eigenvalues. By solving this polynomial equation for λ , we obtain the eigenvalues of L . For each eigenvalue λ_i , substitute it back into the equation $(L - \lambda_i I)x = 0$ to solve for the eigenvector x . This is a system of linear equations, and its solution will give the corresponding eigenvector for λ_i .

Question: Compute the eigenvalues and eigenvectors of L . Based on the computed eigenvectors, provide the Laplacian positional encoding for the nodes in the graph. (Hint: the eigenvectors are normalized to construct the Laplacian PE.)

3. **Random Walk Structural Encoding.** Considering the same graph as above, **Question:** Compute the transition matrix P for a random walk on the graph, where

$$P = D^{-1}A$$

4. **Stationary Distribution** The stationary distribution of a random walk on a graph is a probability distribution across the nodes such that it remains unchanged after applying the transition matrix P . Mathematically, if π is the stationary distribution, then:

$$\pi P = \pi \tag{5}$$

Where the norm of π is 1. **Question:** Given the transition matrix P you just got, compute the stationary distribution of a random walk on the graph.

5. **Question:** Explain how the length of a random walk (number of steps) can influence the structural information captured about a node. Also, explain the relationships between the stationary probability and the degree of the node.

References

- [1] Jiahong Liu, Menglin Yang, Min Zhou, Shanshan Feng, and Philippe Fournier-Viger. Enhancing hyperbolic graph embeddings via contrastive learning, 2022.
- [2] Wei Xia, Tianxiu Wang, Quanyue Gao, Ming Yang, and Xinbo Gao. Graph embedding contrastive multi-modal representation learning for clustering. *IEEE Transactions on Image Processing*, 32:1170–1183, 2023.